Lab: Understanding Mock Objects

Name: Zaw Thant Khaing **Student ID:** 6805140017

Goal

In this lab, you will learn how Python's `Mock` object works by:

1. Running a test.
2. Observing the result.
3. Changing one line of code.
4. Running the test again.
5. Explaining what changed.

By the end, you should understand:

- How a basic mock behaves
- What `return_value` does
- What `side_effect` does
- When to use `return_value`
- When to use `side_effect`

Exercise 1: Basic Mock Object

File: `test_mock_basics.py`

```
from unittest.mock import Mock

def test_mock_basics():
    mock = Mock()

    result = mock.some_method()

    print(result)

    assert isinstance(result, Mock)
```

Run

```
pytest test_mock_basics.py -v -s
```

Observe

Notice that:

```
mock.some_method()
```

works even though `some_method()` was never defined.

The return value is another Mock object.

Experiment A

Change:

```
mock.some_method()
```

to

```
mock.anything()
```

Run again.

Question

Why does the test still pass?

The test still passes because Mock allows us to call methods that we did not create. It returns another Mock object.

Experiment B

Change:

```
mock.anything()
```

to

```
mock.whatever(1, 2, 3)
```

Run again.

Question

Why does Mock allow methods that do not exist?

Mock can create methods automatically when we call them. We do not need to define them first.

Exercise 2: return_value

File: `test_return_value.py`

```
from unittest.mock import Mock

def test_return_value():
    mock = Mock()

    mock.get_status.return_value = "OK"

    assert mock.get_status() == "OK"
```

Run

```
pytest test_return_value.py -v
```

Experiment A

Change:

```
"OK"
```

to

```
"FAILED"
```

Run again.

Question

What changed?

The mock returns "FAILED" instead of "OK".
The test fails because it still expects "OK".

Experiment B

Add:

```
print(mock.get_status())
print(mock.get_status())
print(mock.get_status())
```

before the assertion.

Question

Why is the same value returned every time?

Because return_value gives the same value every time we call the method.

Experiment C

Change:

```
mock.get_status.return_value = "OK"
```

to

```
mock.get_status.return_value = 123
```

Question

Does Mock care about the data type?

No. Mock can return different types of data, such as a string or a number. Here, it returns 123.

Exercise 3: side_effect as Exception

File: `test_side_effect_exception.py`

```
from unittest.mock import Mock
import pytest

def test_side_effect_exception():
    mock = Mock()

    mock.login.side_effect = ValueError(
        "Invalid credentials"
    )

    with pytest.raises(ValueError):
        mock.login()
```

Run

```
pytest test_side_effect_exception.py -v
```

Experiment A

Change:

```
ValueError
```

to

```
RuntimeError
```

Run again.

Question

Why does the test fail?

The mock raises a RuntimeError, but the test expects a ValueError. They are different errors.

Experiment B

Change:

```
with pytest.raises(ValueError):
```

to

```
with pytest.raises(RuntimeError):
```

Run again.

Question

What is pytest checking?

Pytest checks if the code raises the expected error.

If the error is correct, the test passes.

Experiment C

Remove:

```
with pytest.raises(...)
```

Run again.

Question

What happens when the exception is not handled?

With side_effect, the mock returns "A", then "B", then "C".

With return_value, it returns the whole list every time.

Exercise 4: side_effect as a List

File: `test_side_effect_list.py`

```
from unittest.mock import Mock

def test_side_effect_list():
    mock = Mock()

    mock.get_data.side_effect = [
        "A",
        "B",
        "C"
    ]

    assert mock.get_data() == "A"
    assert mock.get_data() == "B"
    assert mock.get_data() == "C"
```

Run

```
pytest test_side_effect_list.py -v
```

Experiment A

Add:

```
print(mock.get_data())
```

after the last assertion.

Run again.

Question

What exception is raised?

A StopIteration error is raised because there are no more values in the list.

Experiment B

Change:

```
["A", "B", "C"]
```

to

```
[1, 2]
```

Question

How many successful calls can be made?

Two calls can be successful because the list has only two values: `1` and `2`.

Experiment C

Replace:

```
side_effect
```

with

```
return_value
```

Question

What changes in behavior?

With side_effect, the mock returns "A", then "B", then "C".

With return_value, it returns the whole list every time.

Exercise 5: side_effect as Function

File: `test_side_effect_function.py`

```
from unittest.mock import Mock

def double(x):
    return x * 2

def test_side_effect_function():
    mock = Mock()

    mock.calculate.side_effect = double

    assert mock.calculate(5) == 10
```

Run

```
pytest test_side_effect_function.py -v
```

Experiment A

Change:

```
return x * 2
```

to

```
return x * 3
```

Run again.

Question

Why does the assertion fail?

The function now returns 15 instead of 10. The test expects 10, so it fails.

Experiment B

Change:

```
assert mock.calculate(5) == 10
```

to

```
assert mock.calculate(5) == 15
```

Run again.

Question

What does this prove about `side_effect`?

It shows that side_effect can run a function and return its result.

Experiment C

Replace:

```
mock.calculate.side_effect = double
```

with

```
mock.calculate.return_value = 10
```

Question

What information is lost?

We lose the calculation. return_value = 10 always returns 10, even if we give it a different number.

Exercise 6: return_value vs side_effect

File: `test_comparison.py`

```
from unittest.mock import Mock

def test_comparison():
    mock = Mock()

    mock.fetch.return_value = "STATIC"

    assert mock.fetch() == "STATIC"
    assert mock.fetch() == "STATIC"
    assert mock.fetch() == "STATIC"
```

Run

```
pytest test_comparison.py -v
```

Experiment A

Replace `return_value` with `side_effect` and use:

```
mock.fetch.side_effect = [
    "A",
    "B",
    "C"
]
```

Question

Which calls now return different values?

The first call returns "A", the second returns "B", and the third returns "C".

Experiment B

Replace:

```
side_effect = ["A", "B", "C"]
```

with

```
side_effect = RuntimeError("Boom")
```

Question

Can `return_value` simulate this behavior?

Why or why not?

No. return_value returns a fixed value. side_effect can raise an error when we call the mock.

Experiment C

Replace:

```
side_effect = RuntimeError("Boom")
```

with

```
def fetch(value):
    return value.upper()

mock.fetch.side_effect = fetch
```

Then call:

```
print(mock.fetch("hello"))
```

Question

What can a function-based `side_effect` do that `return_value` cannot?

It can use the input to make a new result. For example, it can change "hello" to "HELLO". return_value always returns the same value.

Final Reflection

Complete the following:

Use `return_value` when:

We want the same value every time.

We want to set a fixed result for testing.

We do not need to run a function.

Use `side_effect` when:

We want to test errors.

We want different values on different calls.

We want to run a function or do calculations.

One-sentence summary:

return_value gives the same value every time, while side_effect can give different values, raise errors, or run a function.

Key Takeaways

```
return_value        = same result every call
```

```
side_effect (exception)  = simulate failures
```

```
side_effect (list)       = simulate changing responses
```

```
side_effect (function)   = simulate custom logic
```

Rule of Thumb

Need the same answer every time? → `return_value`

Need exceptions, sequences, or custom logic? → `side_effect`